

Fast LeWorldModel

Yuntian Gao, Xiangyu Xu[†]

Xi'an Jiaotong University

[†]Corresponding author

Joint-Embedding Predictive Architectures (JEPAs), including recent LeWorldModel (LeWM), have become a promising foundation for reconstruction-free visual world models. For visual planning, however, LeWM evaluates candidate action sequences by repeatedly applying a local one-step latent transition model. This autoregressive rollout makes planning computationally expensive and exposes the predicted trajectory to accumulated latent errors as the horizon grows. We propose *Fast LeWorldModel* (Fast-LeWM), a fast latent world model that replaces repeated local rollout with action-prefix prediction. Given the current latent and a candidate action sequence, Fast-LeWM encodes its prefixes and predicts the future latents reached after executing those prefixes in parallel. By making action prefixes the basic prediction unit, Fast-LeWM directly models action effects accumulated to different extents over multiple horizons. This prefix-level supervision forces the model to learn how states continuously evolve under different action prefixes, rather than only fitting one-step state transitions. During planning, the predictor can use the last prefix token from the encoded action sequence to evaluate the corresponding future latent without explicitly rolling through each intermediate imagined state. Across multiple tasks, Fast-LeWM improves average success over LeWM while substantially reducing planning time, achieving lower open-loop latent loss whose growth becomes significantly slower as the rollout horizon increases.

Code: <https://github.com/Yuntian-Gao/Fast-LeWorldModel>

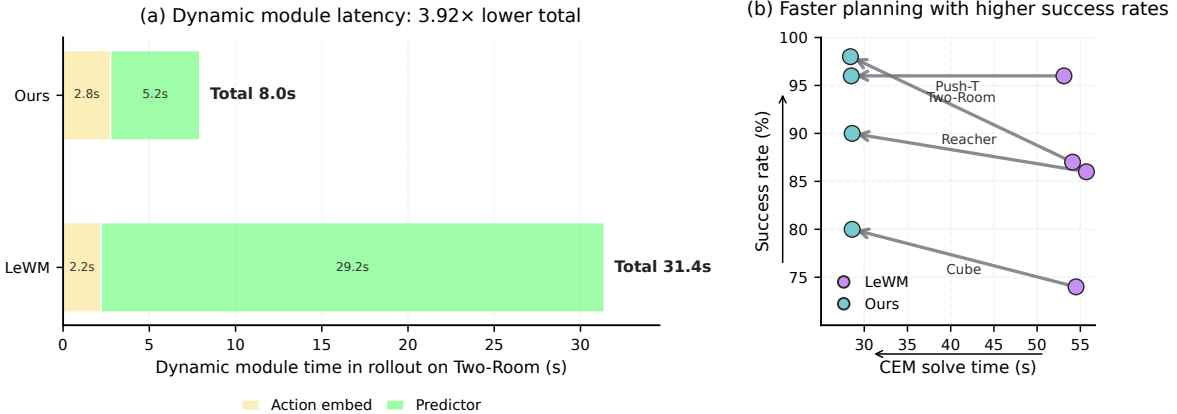


Figure 1 Planning efficiency and success rate across tasks. (a) Runtime breakdown of the two dominant rollout modules on Two-Room. Our method reduces the dynamics-module time in rollout, including action encoding and latent prediction, from 31.4s to 8.0s by replacing autoregressive latent rollout with parallel prefix prediction. (b) Full CEM planning results on four tasks. Each arrow connects LeWM Maes et al. (2026) to our method under the same planning protocol, showing that our method consistently reduces CEM solve time from about 54s to about 28s while improving or maintaining task success.

1 Introduction

World models enable agents to plan by predicting the consequences of their actions before acting Ha & Schmidhuber (2018); Hafner et al. (2019, 2020). For visual planning, reconstruction-free JEPA-style world models learn to predict future embeddings rather than pixels, and recent systems such as LeWorldModel

(LeWM) show that latent prediction can support reward-free goal-conditioned planning from pixels [LeCun et al. \(2022\)](#); [Assran et al. \(2023\)](#); [Bardes et al. \(2024\)](#); [Sobal et al. \(2025\)](#); [Zhou et al. \(2025\)](#); [Maes et al. \(2026\)](#).

Despite this progress, LeWM-style planning remains costly and susceptible to accumulated prediction error. Its dynamics model is inherently local: it predicts only the next latent state $\hat{z}_{t+1}$ from the current latent z_t and action a_t . To evaluate a candidate action sequence, the planner must therefore apply this one-step model autoregressively, repeatedly feeding predicted latents back into the dynamics model G_{LeWM} , and the terminal prediction $\hat{z}_{t+k}$ is obtained by composing local transitions:

$$\hat{z}_{t+k} = G_{\text{LeWM}}(G_{\text{LeWM}}(\cdots G_{\text{LeWM}}(z_t, a_t), \dots), a_{t+k-1}).$$

where each prediction depends on the intermediate imagined latents produced along the rollout.

This creates two practical limitations. First, candidate evaluation becomes slow because autoregressive rollout must generate the entire imagined latent trajectory step by step, repeating action encoding and latent prediction. Second, errors introduced at early or intermediate imagined states can propagate into later predictions, making rollout increasingly unreliable as the horizon grows.

We propose **Fast LeWorldModel** (Fast-LeWM), a fast latent world model built around action-prefix prediction. Instead of advancing the latent state one step at a time, Fast-LeWM predicts the latents reached after executing the prefixes of an encoded action sequence, allowing the model to evaluate these prefix outcomes directly and in parallel. Given the current visual latent z_t and an action prefix

$$a_{t:t+k-1} = (a_t, \dots, a_{t+k-1}),$$

Fast-LeWM predicts the future latent reached after executing that prefix:

$$\hat{z}_{t+k} = G_{\text{Fast-LeWM}}(z_t, a_{t:t+k-1}), \quad k = 1, \dots, H.$$

Rather than estimating a long trajectory through repeated adjacent transitions, Fast-LeWM directly predicts the latent outcomes of different action prefixes from the observed anchor latent. Different prefixes contain different degrees of accumulated action effects and correspond to different future latents, so prefix prediction gives the dynamics model a direct interface for multi-horizon state evolution.

To implement this interface efficiently, Fast-LeWM uses an action-prefix encoder and a parallel latent predictor. The action-prefix encoder processes a candidate action sequence with a causal mask and converts it into multiple prefix tokens, where the token at position k summarizes only $a_t, \dots, a_{t+k-1}$ together with the current latent context. A parallel predictor then maps the current latent and all prefix tokens to their corresponding future latents in one forward pass. Dense prefix-level supervision ties each prefix token to the future latent reached after executing the corresponding partial action sequence. Through this training objective, the model learns to predict in parallel how the state evolves under different action prefixes, rather than only modeling local one-step state evolution.

This design naturally benefits planning. At planning time, the predictor takes all action prefixes as input and predicts all corresponding future states in parallel. These state predictions do not depend on one another sequentially, so prediction errors are not recursively accumulated inside the predictor. This reduces accumulated rollout error and accelerates state-evolution prediction through parallel prefix prediction. As shown in [Figure 1](#), Fast-LeWM substantially reduces the dynamics-module cost inside CEM while improving or preserving task success. The proposed method also lowers open-loop prediction error and slows its growth over longer horizons.

Our contributions are:

- We identify the local one-step transition interface in LeWM as a key bottleneck, leading to slow autoregressive rollouts and the accumulation of latent prediction errors over long horizons.
- We propose Fast-LeWM, a fast latent world model that reformulates latent dynamics modeling from single-step transitions to action-prefix prediction, which enables dense prefix-level supervision and turns accumulated action effects into direct dynamic training targets.

- We evaluate Fast-LeWM on all planning tasks from LeWM under the same protocol. Fast-LeWM improves the average success rate from 85.8% to 90.5%, accelerates the dynamics module by $3.9\times$ (31.4s to 8.0s), reduces the full CEM solve time by 48.0% (54.4s to 28.3s), and substantially lowers both open-loop prediction error and its growth over the long horizon.

2 Related Work

Latent World Models for Planning. World models learn action-conditioned dynamics that allow agents to predict the consequences of candidate actions before execution [Ha & Schmidhuber \(2018\)](#); [Hafner et al. \(2019, 2020\)](#). For pixel-based control, a common approach is to encode observations into compact latent states and perform planning or policy learning in the learned latent space. PlaNet and Dreamer learn latent dynamics from image observations and use imagined trajectories for planning or behavior learning [Hafner et al. \(2019, 2020, 2025\)](#), while TD-MPC and TD-MPC2 show that decoder-free latent dynamics can support efficient model-predictive control in continuous-control domains [Hansen et al. \(2022, 2024\)](#). In offline reward-free goal-conditioned planning, learned latent dynamics can be combined with test-time trajectory optimization, such as CEM, to search for action sequences whose predicted future latent matches a goal latent [Rubinstein & Kroese \(2004\)](#); [Sobal et al. \(2025\)](#); [Zhou et al. \(2025\)](#); [Maes et al. \(2026\)](#). This setting makes the dynamics-query interface especially important: the model is evaluated online for many sampled candidate sequences, so sequential rollout can dominate planning time and can expose long-horizon predictions to accumulated latent error.

Reconstruction-Free Visual Dynamics. Many visual world models learn representations through reconstruction, but pixel reconstruction can force the latent state to preserve visual details that are not necessarily relevant for control. Joint-Embedding Predictive Architectures (JEPAs) provide a reconstruction-free alternative by predicting future or masked embeddings rather than pixels [LeCun et al. \(2022\)](#); [Assran et al. \(2023\)](#); [Bardes et al. \(2024\)](#). This idea has recently been adopted for visual planning: PLDM trains a latent dynamics model from reward-free offline trajectories using a JEPA-style objective with collapse-prevention regularization [Bardes et al. \(2022\)](#); [Sobal et al. \(2022, 2025\)](#); DINO-WM predicts frozen DINOv2 patch features, avoiding end-to-end representation collapse by relying on a pretrained visual encoder [Oquab et al. \(2024\)](#); [Zhou et al. \(2025\)](#); and LeWorldModel (LeWM) trains a compact end-to-end JEPA world model from raw pixels using a next-embedding prediction loss together with SIGReg-style Gaussian regularization [Balestriero & LeCun \(2025\)](#); [Maes et al. \(2026\)](#).

Rather than introducing a new visual representation objective, we change how candidate action sequences are evaluated by the dynamics model. LeWM-style planning predicts the next latent locally and obtains a multi-step prediction by applying the transition model sequentially, so later predictions depend on earlier predicted latents. In contrast, Fast-LeWM uses action prefixes as multi-horizon queries, conditioning each future latent on the observed anchor latent and the corresponding action prefix rather than solely on the immediately preceding imagined latent, which mitigates the compounding dependence of sequential rollout.

3 Method

3.1 Reward-Free Latent World Models

We consider an offline, reward-free dataset of observation-action trajectories

$$\mathcal{D} = \{\tau^{(n)}\}_{n=1}^N, \quad \tau = \{(o_t, \mathbf{a}_t)\}_{t=1}^T, \quad (1)$$

where o_t is a pixel observation and $\mathbf{a}_t \in \mathbb{R}^{d_a}$ is a continuous action. The goal is to learn a latent world model that supports goal-conditioned planning without reward labels. A visual encoder maps observations into latent embeddings,

$$\mathbf{z}_t = f_\theta(o_t), \quad \mathbf{z}_t \in \mathbb{R}^d, \quad (2)$$

and a goal observation o_g is similarly encoded as $\mathbf{z}_g = f_\theta(o_g)$. Planning then searches for an action sequence whose predicted future latent is close to $\mathbf{z}_g$.

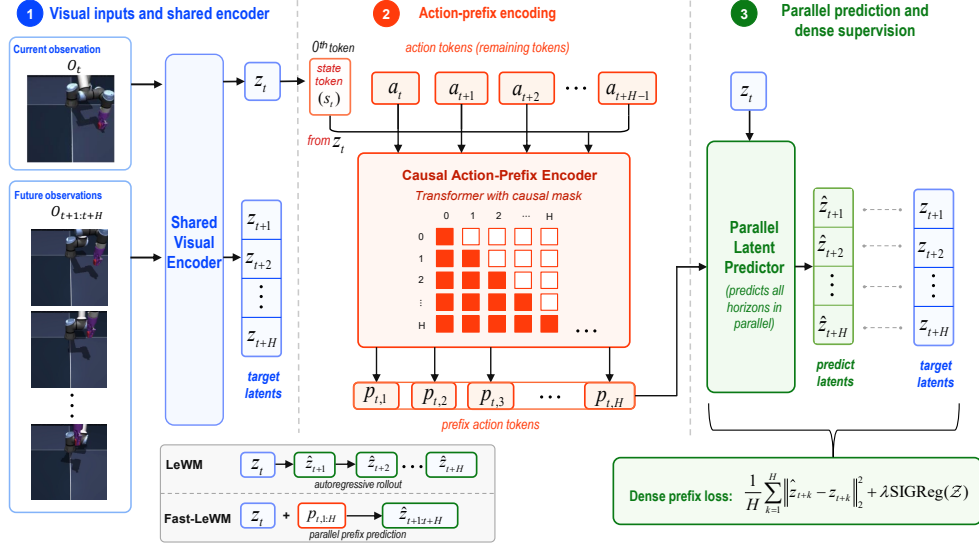


Figure 2 Fast-LeWM training pipeline. The current observation o_t is encoded into a latent state z_t . An Action-Prefix Encoder takes a state-action token sequence formed by prepending a state token mapped from z_t as the 0-th token to the action tokens $(a_t, \dots, a_{t+H-1})$. Under causal masking, the action-position outputs form prefix tokens $(p_{t,1}, \dots, p_{t,H})$, where $p_{t,k}$ corresponds to the prefix $(a_t, \dots, a_{t+k-1})$, while the 0-th output token is discarded. Using the prefix tokens together with the anchor latent, the Parallel Predictor directly generates all future latents $(\hat{z}_{t+1}, \dots, \hat{z}_{t+H})$ in one forward pass, supervised by encoded future observations through a dense multi-horizon loss. Unlike isolated one-step transition learning, dense action-prefix prediction supervises multiple horizons directly, forcing the model to learn the multi-horizon state evolution induced by an action sequence while reducing error propagation into later horizons.

In this setting, the dynamics model defines how candidate action sequences are evaluated in latent space.

3.2 Autoregressive Rollout in LeWorldModel

A common latent world-model design, including LeWM-style dynamics, learns a local state-transition predictor

$$\hat{\mathbf{z}}_{t+1} = F_\phi(\mathbf{z}_t, \mathbf{a}_t), \quad (3)$$

trained with a next-latent prediction loss

$$\mathcal{L}_{\text{pred}}^{\text{1step}} = \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2^2. \quad (4)$$

Following reconstruction-free JEPA-style world models, an anti-collapse regularizer such as SIGReg is added over a batch of latent embeddings Z :

$$\mathcal{L}_{\text{1step}} = \mathcal{L}_{\text{pred}}^{\text{1step}} + \lambda \text{SIGReg}(Z). \quad (5)$$

Given a candidate sequence $\mathbf{a}_{t:t+H-1}$, a one-step model estimates the terminal latent by autoregressive rollout,

$$\hat{\mathbf{z}}_{t+k} = F_\phi(\hat{\mathbf{z}}_{t+k-1}, \mathbf{a}_{t+k-1}), \quad k = 1, \dots, H, \quad (6)$$

and scores the sequence by

$$C_{\text{AR}}(\mathbf{a}_{t:t+H-1}) = \|\hat{\mathbf{z}}_{t+H} - \mathbf{z}_g\|_2^2. \quad (7)$$

The main drawback of this interface is the long autoregressive chain required during candidate evaluation. Even when the planner ultimately scores only selected future states, reaching those states requires generating the preceding imagined latents $\hat{\mathbf{z}}_{t+1}, \dots, \hat{\mathbf{z}}_{t+H-1}$ and repeatedly invoking the action encoder and latent transition modules. This repeated rollout becomes expensive when CEM evaluates many candidate sequences. Moreover, intermediate predicted latents are introduced early into the computation and then reused as inputs for later predictions, so approximation errors can be injected repeatedly and accumulate along the rollout horizon.

3.3 Fast LeWorldModel

Fast-LeWM addresses these limitations by reducing the sequential dependence among predicted future states. Instead of producing $\hat{\mathbf{z}}_{t+1}, \dots, \hat{\mathbf{z}}_{t+H}$ through an autoregressive chain, our key idea is to predict each future state directly from the observed anchor latent and the action prefix that leads to that state. Under this view, different future states are not generated by feeding one predicted state into the next; they can be queried independently once their corresponding action prefixes are encoded. This motivates the design of an action-prefix encoder together with a parallel latent predictor. Figure 2 gives an overview of the resulting training pipeline.

For a candidate action sequence $\mathbf{a}_{t:t+H-1}$, each horizon k corresponds to the prefix

$$\mathbf{a}_{t:t+k-1} = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+k-1}), \quad k = 1, \dots, H. \quad (8)$$

The target associated with this prefix is the latent state reached after executing these k actions, namely $\mathbf{z}_{t+k}$. Fast-LeWM maps each prefix to a learned prefix token

$$\mathbf{p}_{t,k} = E_{\psi}^{(k)}(\mathbf{a}_t, \dots, \mathbf{a}_{t+k-1}), \quad (9)$$

where $\mathbf{p}_{t,k}$ summarizes the accumulated effect of the first k actions. The latent predictor then uses the anchor latent and each prefix token to predict the corresponding future latent:

$$\hat{\mathbf{z}}_{t+k} = G_{\phi}(\mathbf{z}_t, \mathbf{p}_{t,k}), \quad k = 1, \dots, H. \quad (10)$$

This design has two direct benefits. During training, dense supervision over all prefix tokens forces the model to learn how action effects accumulate over different horizons, rather than only fitting local one-step state evolution. During rollout, all action prefixes can be processed together and their corresponding future states can be generated in parallel. Because these predictions are anchored at the observed latent $\mathbf{z}_t$ and do not depend sequentially on one another, the model reduces recurrent error accumulation while accelerating rollout through parallel state-evolution prediction.

3.4 Action-Prefix Encoder

The role of the action-prefix encoder is to output horizon-specific prefix tokens $\mathbf{p}_{t,1:H}$, where the token at horizon k corresponds only to the action prefix of length k , while avoiding leakage from future actions into shorter prefixes. Given an action sequence

$$\mathbf{a}_{t:t+H-1} = (\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}), \quad (11)$$

the encoder outputs a dense sequence of prefix tokens

$$\mathbf{p}_{t,1:H} = E_{\psi}(\mathbf{a}_{t:t+H-1}) = (\mathbf{p}_{t,1}, \mathbf{p}_{t,2}, \dots, \mathbf{p}_{t,H}). \quad (12)$$

We instantiate E_{ψ} as a causal Transformer over action tokens. With the causal mask, the representation at horizon k can attend only to actions $\mathbf{a}_t, \dots, \mathbf{a}_{t+k-1}$, yielding

$$\mathbf{p}_{t,k} = E_{\psi}^{(k)}(\mathbf{a}_t, \dots, \mathbf{a}_{t+k-1}), \quad k = 1, \dots, H. \quad (13)$$

In practice, an action prefix does not determine its outcome by itself. The same open-loop actions may move the agent toward different regions, produce different contacts, or have different object effects depending on the current configuration of the scene. To provide this context with minimal overhead, we map the current latent $\mathbf{z}_t$ through a lightweight MLP into a state token and prepend it as the 0-th token before the action-token sequence. With this state token, the implemented encoder can be viewed as the conditioned form

$$\mathbf{p}_{t,k} = E_{\psi}^{(k)}(\mathbf{a}_t, \dots, \mathbf{a}_{t+k-1} \mid \mathbf{z}_t). \quad (14)$$

3.5 Parallel Latent Predictor

Given the current latent and prefix tokens, the predictor estimates future latents for all prefix horizons in parallel:

$$\hat{\mathbf{z}}_{t+1:t+H} = G_\phi(\mathbf{z}_t, \mathbf{p}_{t,1:H}). \quad (15)$$

Equivalently,

$$\hat{\mathbf{z}}_{t+k} = G_\phi(\mathbf{z}_t, \mathbf{p}_{t,k}), \quad k = 1, \dots, H. \quad (16)$$

The predictor therefore uses each prefix token to specify which accumulated action effect should be applied to the anchor latent. Because every horizon is predicted from its own prefix token, the model learns prefix-level state evolution rather than only propagating a local one-step change. This design is also computationally favorable for planning: all queried horizons share one action-prefix encoding pass and one parallel latent-prediction pass, without repeatedly iterating one-step next-state predictions, which helps reduce error accumulation.

3.6 Dense Prefix Prediction Objective

For a training segment

$$(o_t, \mathbf{a}_t, o_{t+1}, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}, o_{t+H}), \quad (17)$$

we encode the current and future observations as

$$\mathbf{z}_{t+i} = f_\theta(o_{t+i}), \quad i = 0, \dots, H. \quad (18)$$

The action-prefix encoder and predictor produce prefix-level predictions $\hat{\mathbf{z}}_{t+1:t+H}$. Every action prefix receives its own latent target:

$$\mathcal{L}_{\text{prefix}} = \frac{1}{H} \sum_{k=1}^H \|\hat{\mathbf{z}}_{t+k} - \mathbf{z}_{t+k}\|_2^2. \quad (19)$$

This dense objective supervises not only the terminal outcome but also the intermediate states induced by partial action prefixes, forcing the model to learn how the latent state evolves as more actions are appended to the sequence.

We retain the SIGReg regularizer used by reconstruction-free latent world models to prevent latent collapse:

$$\mathcal{L}_{\text{AP}} = \mathcal{L}_{\text{prefix}} + \lambda \text{SIGReg}(Z). \quad (20)$$

3.7 Planning with Action Prefixes and Self-Consistency

During planning, we use the same CEM-based goal-conditioned latent planning protocol as LeWM and score candidate action sequences by their terminal latent distance to the goal. Let m index a candidate action sequence sampled by CEM, LeWM reaches $\hat{\mathbf{z}}_{t+H}^{(m)}$ by repeatedly applying a one-step dynamics model along imagined latents. In contrast, Fast-LeWM treats the action prefix as the rollout unit, so the corresponding prefix token $\mathbf{p}_{t,H}^{(m)}$ provides a direct path from the current latent to that future latent.

The basic candidate cost is then

$$C_{\text{goal}}^{(m)} = \left\| \hat{\mathbf{z}}_{t+H}^{(m)} - \mathbf{z}_g \right\|_2^2. \quad (21)$$

CEM updates its sampling distribution using the lowest-cost candidates, and the selected action sequence is executed until the next decision point. Thus, Fast-LeWM keeps the same planning objective as LeWM but changes the rollout interface.

This prefix-level interface also provides an optional self-consistency signal during planning. In addition to predicting the terminal latent directly from the length- H prefix, the model can take one intermediate prefix step and then predict the remaining horizon from the intermediate latent, yielding another terminal estimate $\tilde{\mathbf{z}}_{t+H}^{(m)}$. The discrepancy between these two terminal estimates can be added as a β -weighted model-consistency penalty:

$$C^{(m)} = C_{\text{goal}}^{(m)} + \beta \left\| \hat{\mathbf{z}}_{t+H}^{(m)} - \tilde{\mathbf{z}}_{t+H}^{(m)} \right\|_2^2, \quad (22)$$

Here $\beta \geq 0$ controls the strength of the optional consistency term. Setting $\beta = 0$ recovers the goal-only CEM objective, while larger values encourage CEM to prefer candidates whose terminal predictions are stable under different prefix decompositions.

Table 1 Planning success rate (%) across environments. Fast-LeWM uses the same planning objective as LeWM and improves the average success rate from 85.8% to 90.5%. The optional self-consistency term provides an auxiliary model-consistency signal during candidate scoring and further improves the average success rate to 92.0%.

Method	Two-Room	Reacher	PushT	OGBench-Cube	Avg.
PLDM Sobal et al. (2025)	97	78	78	65	79.5
DINO-WM Zhou et al. (2025)	100	79	74	86	84.8
LeWM Maes et al. (2026)	87	86	<u>96</u>	74	85.8
Fast-LeWM	<u>98</u>	<u>88</u>	<u>96</u>	80	<u>90.5</u>
Fast-LeWM+Self-Consistency	<u>98</u>	90	98	<u>82</u>	92.0

4 Experiments

4.1 Experimental Setup

Environments and datasets. We follow the evaluation protocol of LeWM and use the same offline datasets, observation preprocessing, and goal-conditioned planning setup. The environments include Two-Room [Sobal et al. \(2025\)](#) / PushT [Zhou et al. \(2025\)](#) / Reacher [Tassa et al. \(2018\)](#) / OGBench-Cube [Park et al. \(2025\)](#).

Planning protocol. At test time, we follow the same goal-conditioned latent planning protocol as LeWM. Following LeWM, we use the same frame-skipped control interface with an action skip of 5 and set the planning horizon to $H = 5$. Thus, each planned action is executed for five primitive environment steps, and the full planning horizon covers 25 environment steps. The optimized action sequence is executed following the same MPC schedule as LeWM.

For the self-consistency variant in Table 1, we set $\beta = 1$ and add a consistency loss between directly predicting the 25-step latent and predicting it via an intermediate 10-step latent.

Model architecture. Our visual encoder and anti-collapse regularization follow LeWM. The main change is the dynamics module. The action-prefix encoder is implemented as a causal Transformer over a state-action token sequence. The first token is obtained by mapping the current latent z_t through a two-layer MLP with hidden width 768, and the remaining tokens are embeddings of the future actions $(a_t, \dots, a_{t+H-1})$. Both the state token and action-prefix token dimensions are set to 192. The action-prefix Transformer has 3 layers and 6 attention heads, with per-head dimension 32. We use sinusoidal positional encodings over the state-action token sequence, with sine and cosine functions at exponentially spaced frequencies.

For each prefix token $p_{t,k}$, the latent dynamics predictor modulates the initial latent z_t with the prefix representation and maps it to $\hat{z}_{t+k}$ using a 6-layer action-modulated residual MLP. The predictor uses latent dimension 192, hidden width 2048, fusion width 768, AdaLN-zero modulation, and dropout 0.1. The resulting model has 17.9M parameters, comparable to the 18.0M parameters of the released LeWM checkpoint.

Training details. We train Fast-LeWM with batch size 128 on all environments except Cube, where we use batch size 32 for more stable and faster convergence. All models are trained for 10 epochs, matching the LeWM protocol. During training, the prediction horizon is determined adaptively from the loaded trajectories and clamped to the range $[1, 5]$. All other training hyperparameters, data preprocessing, image resolution, optimizer settings, and evaluation settings follow LeWM unless otherwise noted. Because Fast-LeWM anchors every prediction at the current observed latent z_t , it does not rely on a visual history; therefore, we set the history size to 1 for all environments.

4.2 Planning Performance and Efficiency

Tables 1 and 2 compare Fast-LeWM with LeWM under the same planning protocol. Table 1 shows that Fast-LeWM achieves higher or equal success rates across all environments, increasing the average success rate from 85.8% to 90.5%. Adding the optional self-consistency term further improves the average success rate to

Table 2 Planning efficiency under the same CEM budget on Two-Room. Dynamics time includes action encoding and latent prediction only. Times are measured on a single NVIDIA 4090.

Method	Model calls	Dynamics time	CEM time
LeWM	5	31.4s	54.4s
Fast-LeWM	1	8.0s	28.3s

92.0%, suggesting that consistency across alternative prefix-based terminal estimates can provide a useful auxiliary signal for candidate selection.

Table 2 shows that the improved success rate also comes with substantially lower planning cost. Unlike LeWM, which applies its dynamics predictor sequentially across the planning horizon, Fast-LeWM evaluates each candidate sequence using one action-prefix encoding pass followed by parallel latent prediction. We report both the dynamics-evaluation time, which includes action encoding and latent prediction, and the full CEM solve time. As in LeWM, the latter additionally includes goal- and observation-image encoding, score computation, and other data-operation overhead. Fast-LeWM reduces dynamics-evaluation time from 31.4s to 8.0s and CEM solve time from 54.4s to 28.3s. Because all tasks use the same image resolution and latent dimensionality, their dynamics-evaluation costs are nearly identical to Two-Room. Overall, Fast-LeWM improves planning accuracy while directly reducing the repeated dynamics-evaluation cost that dominates sampling-based latent planning.

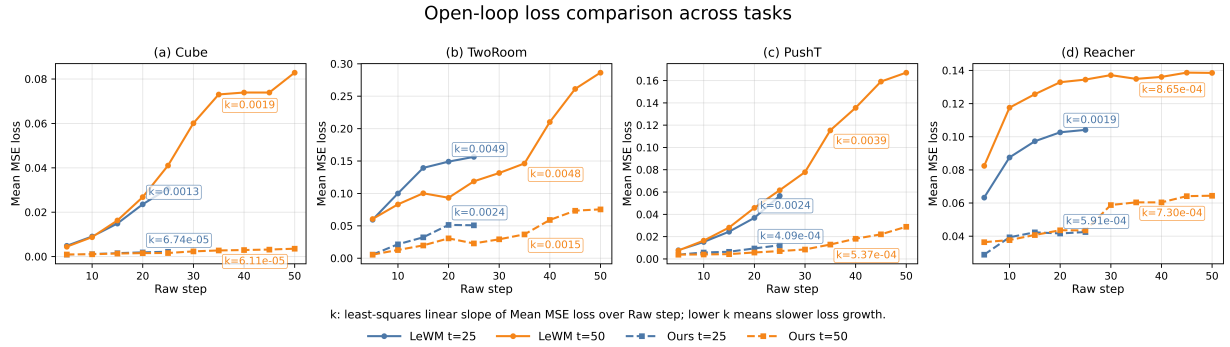


Figure 3 Open-loop latent prediction loss across four tasks. The loss is measured in latent space against the latent states encoded from the corresponding ground-truth future frames. Compared with LeWM, Fast-LeWM consistently reduces both the initial prediction error and the growth rate of error as the state evolves along the real trajectory. This shows that the action-prefix design substantially mitigates open-loop error accumulation and provides a more error-resistant interface for modeling continuous state transitions over multiple horizons.

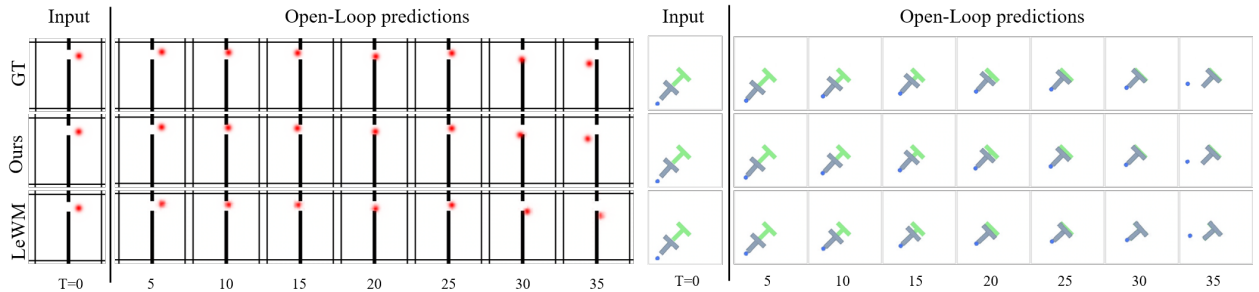


Figure 4 Predictor rollouts on Two-Room and PushT, visualized by decoding predicted latents. Fast-LeWM exhibits smaller open-loop prediction error than LeWM because its action-prefix design predicts target horizons without repeatedly iterating one-step next-state predictions, which helps reduce error accumulation.

Table 3 Physical latent probing results on PushT.

Property	Model	Linear		MLP	
		MSE ↓	r ↑	MSE ↓	r ↑
Agent Loc.	PLDM	0.090	0.955	0.014	0.993
	LeWM	0.052	0.974	0.004	0.998
	Ours	0.048	0.976	0.001	1.000
Block Loc.	PLDM	0.122	0.938	0.011	0.994
	LeWM	0.029	0.986	0.001	0.999
	Ours	0.029	0.987	0.000	1.000
Block Angle	PLDM	0.446	0.745	0.056	0.972
	LeWM	0.187	0.902	0.021	0.990
	Ours	0.314	0.828	0.009	0.995

4.3 Open-Loop Latent Prediction

We first quantitatively evaluate open-loop prediction in latent space. Given the true initial frame and a future action sequence, each model predicts latent states along the open-loop trajectory, and we plot the latent loss against the corresponding ground-truth future latents over time. Because the planning horizon is $H = 5$ with an action skip of 5, $t = 25$ corresponds to one maximum-horizon prediction for Fast-LeWM, while $t = 50$ requires two such predictions. In contrast, LeWM reaches these horizons by applying its local transition model autoregressively for 5 and 10 steps, respectively. As shown in Figure 3, Fast-LeWM achieves substantially lower initial latent error across all four tasks. We further fit the loss curve with least squares and use its slope to measure how quickly prediction error grows as the state evolves along the real trajectory. The slope of Fast-LeWM is consistently smaller than that of LeWM, showing that action-prefix prediction mitigates open-loop error accumulation and learns a more accurate, continuous model of state evolution.

We also provide a qualitative comparison by visualizing predicted latents with a decoder following LeWM. Both models are conditioned on the initial observation at $t = 0$ and the same future action sequence. As shown in Figure 4, the decoded predictions from Fast-LeWM remain more consistent with the target trajectory, while LeWM exhibits visible drift over longer horizons due to accumulated rollout error.

4.4 Physical State Probing

To further examine the physical information encoded in the learned latent space, we conduct probing experiments on PushT following the protocol of LeWM Maes et al. (2026). We freeze the encoder and train lightweight probes to predict three task-relevant physical variables from latent embeddings: agent location, block location, and block angle. We report both linear and MLP probes, where the linear probe measures direct accessibility of the variables, while the MLP probe evaluates whether the information is retained in a nonlinearly recoverable form.

As shown in Table 3, our method achieves probing performance comparable to LeWM under linear probes and consistently outperforms PLDM. This indicates that our representation preserves a similar level of linearly accessible physical structure. More importantly, our method shows clear advantages under MLP probes across all three variables, achieving the lowest MSE and highest correlation for agent location, block location, and block angle. These results suggest that our latent space retains richer physical state information. We attribute this to the prefix-level training objective: by forcing the model to predict how the state evolves under different action prefixes, rather than only local one-step state evolution, the latent representation must preserve fine-grained physical variables that determine future motion, contact, and object configuration. This encourages the encoder and dynamics module to learn more accurate state-aware features rather than representations that only support local one-step changes.

Table 4 Ablation study across four environments. All variants use the same visual encoder, training protocol, and CEM budget. Success rate (%) is reported for each task.

Variant	Two-Room	Reacher	PushT	Cube
Long-Action LeWM	76	70	80	58
Terminal-only Fast-LeWM	96	80	90	72
Fast-LeWM	98	88	96	80
w/o state token	94	82	92	80

4.5 Ablation Studies

We ablate the components of Fast-LeWM in Table 4. A naive way to speed up LeWM is to enlarge its action block, which we instantiate as *Long-Action LeWM* by changing the original frame-skipped action encoding from 5 to 25 primitive actions. This variant still uses LeWM’s next-state prediction interface: one model step directly predicts the terminal latent after the longer action block. As shown in Table 4, this naive modification performs poorly, indicating that faster planning cannot be achieved by simply making each LeWM transition cover a longer temporal span.

Accordingly, we also remove dense prefix supervision from Fast-LeWM and train a terminal-only variant that supervises only the final latent prediction.

Terminal-only Fast-LeWM still substantially outperforms Long-Action LeWM, showing that action prefixes provide a fast and effective representation for long-horizon rollout. Rather than directly concatenating all actions into one large block, Fast-LeWM represents the sequence through progressively accumulated prefixes, explicitly exposing its internal order structure and cumulative action effects while still producing the terminal prediction in one forward pass. However, terminal-only Fast-LeWM still underperforms the full model because the final-state loss does not explicitly constrain the intermediate prefix tokens to correspond to meaningful partial action outcomes. Dense prefix supervision addresses this by assigning a latent target to each prefix, forcing the action encoder and predictor to model state evolution throughout the sequence rather than only its endpoint.

We also observe better performance when the action-prefix encoder is conditioned on the current state token. This design introduces only a small overhead: a lightweight two-layer MLP maps the current latent into one additional token in the prefix encoder. With this minor cost, the model can interpret the same open-loop action prefix under different initial positions, object configurations, scene geometry, and contact constraints. The state token therefore provides the context needed to disambiguate action effects, improving the predicted effect of each prefix on the future latent state.

5 Conclusion

We introduced Fast-LeWM, a fast latent world model for reward-free visual planning. Instead of relying on autoregressive one-step rollout, Fast-LeWM predicts future latents directly from the current latent and action-prefix representations, enabling parallel multi-horizon prediction. This design reduces repeated dynamics evaluations, mitigates latent error accumulation, and provides dense supervision over continuous state evolution. Experiments across four goal-conditioned planning tasks show that Fast-LeWM improves planning success while substantially reducing CEM solve time and open-loop prediction error. These results suggest that action-prefix prediction is an effective and efficient dynamics interface for latent world-model planning.

References

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

- Randall Balestriero and Yann LeCun. LeJEPa: Provable and scalable self-supervised learning without the heuristics, 2025.
- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv:2404.08471*, 2024.
- David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. *Advances in Neural Information Processing Systems (NeurIPS)*, 31, 2018.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning (ICML)*, 2019.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.
- Nick Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024.
- Nicklas A. Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *International Conference on Machine Learning (ICML)*, 2022.
- Yann LeCun et al. A path towards autonomous machine intelligence, version 0.9.2, 2022-06-27. *OpenReview*, 62(1): 1–62, 2022.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv:2603.19312*, 2026.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations (ICLR)*, 2025.
- Reuven Y Rubinstein and Dirk P Kroese. *The cross-entropy method: a unified approach to combinatorial optimization, Monte-Carlo simulation, and machine learning*, volume 133. Springer, 2004.
- Vlad Sobal, Jyothir SV, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features. *arXiv:2211.10831*, 2022.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *7th Robot Learning Workshop: Towards Robots with Human-Level Abilities*, 2025.
- Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. DeepMind control suite, 2018.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning (ICML)*, 2025.